PROYECTO FINAL

MÉTRICAS DE MANTENIBILIDAD Y EVOLUCIÓN APLICADAS A UNA APLICACIÓN REAL

JUAN SEBASTIAN AGUDELO QUINTERO

DAYANA STEPHANY MOTTA CAMAYO

MÉTRICAS Y ESTÁNDARES DE CALIDAD

JOSE MIGUEL LLANOS MOSQUERA

CORPORACIÓN UNIVERSITARIA DEL HUILA CORHUILA

NEIVA

NOVIEMBRE DE 2026

TABLA DE CONTENIDO:

INTRODUCCIÓN 5
Tema elegido 5
Problema que resuelve 5
Objetivo 6
Alcance 6
REQUISITOS 6
Requisitos funcionales (RF) 7
Requisitos no funcionales (RNF) 9
ARQUITECTURA 10
Estilo seleccionado 10
Justificación 10
Diagramas 12
Estructura del proyecto 13
IMPLEMENTACIÓN 15
Lenguaje y tecnologías 15
Descripción de los módulos 15
Instrucciones de ejecución 16
Pruebas 17
Tabla de trazabilidad 17
DEFINICIÓN DE MÉTRICAS 20
RESULTADOS DE LAS MÉTRICAS 23
Complejidad Ciclomática V(G) 23
Líneas de código (LOC) 24
Suite CK 24
ANÁLISIS Y PLAN DE MEJORAS 27
Interpretación frente a los umbrales 27
Problemas detectados 28
Mejoras propuestas 28
Refactorización implementada 29
CONCLUSIONES Y LECCIONES APRENDIDAS 31

Conclusiones 31
Lecciones aprendidas 32
Reflexión final 33
APORTE INDIVIDUAL 34
Trabajo conjunto 35
Reflexión sobre el trabajo en equipo 36
REFERENCIAS Y ENLACE AL REPOSITORIO 36
Frameworks y librerías del sistema: 36
Herramientas de medición de métricas: 37
Fundamentos teóricos de las métricas: 37
Enlace al repositorio de GitHub 37
Estructura del repositorio: 38
DECLARACIÓN DE AUTORÍA Y USO DE INTELIGENCIA ARTIFICIAL 38
Autoría y originalidad 38
Uso de herramientas de inteligencia artificial 38

LISTADO DE TABLAS:

Tabla 1. Requisitos funcionales (RF) 9
Tabla 2. Requisitos no funcionales (RNF) 9
Tabla 3. Trazabilidad 19
Tabla 4. Complejidad Ciclomática V(G) 23
Tabla 5. Método que supera el umbral V(G) 24
Tabla 6. Método que supera el umbral LOC 24
Tabla 7. WMC mayor a 20 24
Tabla 8. RFC mayor a 50 25
Tabla 9. Resumen general 26
Tabla 10. Problemas detectados 28
Tabla 11. Comparación antes y después 30
Tabla 12. Proyección 31

LISTADO DE FICHAS:

Ficha 1: Complejidad Ciclomática V(G) (McCabe) 21
Ficha 2: Líneas de código (LOC) y porcentaje de comentarios 21
Ficha 3: Suite CK (WMC, DIT, NOC, CBO, RFC, LCOM) 22

LISTADO DE FIGURAS:

Figura 1. Diagrama de componentes. 12
Figura 2. Diagrama de capas. 13
Figura 3. Estructura de carpetas del repositorio, organizada por módulos y capas. 14
Figura 4. Distribución de la complejidad ciclomática V(G) por método. 23
Figura 5. Clases que superan cada umbral de la Suite CK. 25
Figura 6. Comparación de WMC, CBO y RFC de las tres clases más críticas. 26
Figura 7. Contribuciones al repositorio. 36

INTRODUCCIÓN

El presente proyecto nace a partir de la necesidad de poner en práctica todo lo aprendido durante el curso de Métricas y Estándares de Calidad, pero aplicándolo sobre un software real, un desarrollo q funcione de verdad y que pudiéramos analizar a fondo. Para eso decidimos desarrollar el sistema GMS (Gym Management System), una aplicación web que habíamos estado desarrollando a la par para la asignatura de Desarrollo Fullstack y que, con la autorización del docente, decidimos reutilizar y adaptar para este proyecto. La idea no era solo mostrar que la aplicación funciona, sino medir su calidad interna, entender cómo está construida por dentro y proponer mejoras con evidencia.

Tema elegido

Escogimos la gestión de un gimnasio. Parece fácil, pero tiene bastantes piezas: clientes, planes de membresía, pagos, asistencia, entrenadores, rutinas y reportes, y todo debe funcionar en conjunto sin fallar. GMS permite manejar todo esto desde un solo lugar, con cuatro tipos de usuario (administrador, recepcionista, entrenador y cliente), cada uno con sus propios permisos. El backend está hecho en Java con Spring Boot, el frontend en React con TypeScript, la base de datos es PostgreSQL y todo se ejecuta con Docker Compose.

Problema que resuelve

Muchos gimnasios pequeños y medianos siguen trabajando con hojas de cálculo, cuadernos o programas que no están conectados entre sí. Por eso se pierde información de las membresías vencidas, cuesta saber quién entra y quién sale, no queda claro qué entrenador atiende a cada cliente y es difícil sacar reportes en los que se pueda confiar para tomar decisiones. GMS busca resolver esto reuniendo todo en una sola aplicación web con una base de datos relacional, para que la información sea consistente, segura y fácil de rastrear. Así el gimnasio ya no depende de papeles sueltos y puede ver en el momento lo que está pasando.

Objetivo

El objetivo principal es desarrollar una aplicación web que cumpla diez requisitos funcionales y tres no funcionales. Además, queremos medir su calidad interna con métricas de mantenibilidad y evolución, como la complejidad ciclomática, las líneas de código y la suite CK. Con esos resultados vamos a detectar problemas de diseño y proponer mejoras que estén respaldadas por los datos y no por simples opiniones. En resumen, queremos medir nuestro propio software, comparar los resultados con valores de referencia y decidir con evidencia qué conviene mejorar.

Alcance

Para que el proyecto fuera manejable, definimos qué incluir y qué no. Incluimos la autenticación con JWT, la gestión de usuarios y roles, el registro y consulta de clientes, la administración de planes, la creación de membresías, el registro de pagos, el control de asistencia (entrada y salida con código de acceso), la gestión de entrenadores, rutinas y ejercicios, y los reportes de resumen y pagos recientes. También incluimos la interfaz web para cada rol, la base de datos con migraciones versionadas y el despliegue con Docker Compose.

Dejamos por fuera la facturación electrónica, las pasarelas de pago reales, las notificaciones por correo o SMS, la aplicación móvil nativa (la nuestra es web y se adapta a distintas pantallas), el inventario, la reserva de clases grupales, un módulo contable completo y el hardware biométrico (el kiosco funciona con código de acceso). No las incluimos porque el curso no lo pedía y preferimos enfocarnos en medir bien lo que ya habíamos construido.

REQUISITOS

Antes de medir cualquier cosa, necesitábamos tener claro qué debía hacer el sistema y qué tan bien debía hacerlo. Por eso definimos diez requisitos funcionales, que son las cosas que la aplicación tiene que permitir hacer al usuario, y tres requisitos no funcionales, que son las condiciones de calidad que debe cumplir. Cada requisito tiene su identificador, una descripción clara y un criterio de aceptación o de medición, para que no quede en palabras bonitas sino en algo que se pueda comprobar. Además, cada requisito está asociado a un componente del código, para que después podamos rastrear quién lo implementa y con qué prueba se valida.

Requisitos funcionales (RF)

Los diez requisitos funcionales cubren el núcleo del gimnasio: la entrada al sistema, la administración de usuarios, el registro y la consulta de clientes, los planes, las membresías, los pagos, la asistencia, las rutinas y los reportes. La idea era que entre todos formaran el flujo completo de operación de un gimnasio, no solo operaciones sueltas de mantenimiento de datos.

ID	Descripción	Criterio de aceptación	Componente
RF-01	El sistema debe permitir a un usuario iniciar sesión con su nombre de usuario y contraseña, y devolver un token JWT con su rol y fecha de expiración.	Si las credenciales son correctas, el usuario recibe el token y puede acceder a las rutas protegidas. Si son incorrectas, el sistema responde con un error 401 y el mensaje "Invalid credentials".	AuthController, AuthService, JwtService
RF-02	El administrador debe poder crear, listar, consultar, actualizar, activar y desactivar usuarios del sistema, asignándoles uno de los roles: administrador, recepcionista, entrenador o cliente.	Un administrador puede crear un usuario y consultarlo. Un usuario sin rol de administrador recibe un error 403 si intenta gestionar usuarios. El sistema no permite desactivar al administrador del sistema.	UserController, UserService
RF-03	Un administrador o un recepcionista debe poder registrar un nuevo cliente con sus datos personales (documento, nombre completo y contacto), creando al mismo tiempo su usuario de acceso y su membresía inicial.	El cliente queda guardado y se puede consultar por su documento. Si el documento o el nombre de usuario ya existen, el sistema responde con un error 409. Un cliente con rol de miembro no puede registrar a otros clientes (error 403).	MemberController, MemberService
RF-04	El sistema debe permitir consultar el listado de clientes con paginación y filtro por estado (activo o inactivo), ver el detalle de un cliente y actualizar su nombre completo y su contacto.	El listado devuelve una página con metadatos (número de página, tamaño y total de elementos). Si se intenta actualizar un cliente que no existe, el sistema responde con un error 404.	MemberController, MemberService
RF-05	El administrador debe poder crear, listar, consultar, actualizar, activar y desactivar planes de membresía, cada uno con nombre, precio y duración en días.	Si se intenta crear un plan con un nombre que ya está activo, el sistema responde con un error 409. Si el precio es negativo o la duración es menor o igual a cero, la validación lo rechaza con un error 400.	PlanController, PlanService

RF-06	Un administrador o un recepcionista debe poder crear una membresía para un cliente, asociándole un plan activo y una fecha de inicio, y calculando automáticamente la fecha de fin según la duración del plan.	Un cliente solo puede tener una membresía activa al mismo tiempo. Si se intenta crear una segunda, el sistema responde con un error 409. Tampoco se puede crear una membresía con un plan inactivo (error 409). La membresía guarda el precio del plan en el momento de su creación.	MembershipController, MembershipService
RF-07	Un administrador o un recepcionista debe poder registrar pagos parciales o totales sobre una membresía activa, sin que la suma de los pagos supere el precio de la membresía.	Si un pago hace que la suma supere el saldo pendiente, el sistema responde con un error 409 y el mensaje "Payment exceeds membership balance". Si la membresía no está activa, también responde con un error 409. El cliente solo puede consultar su propio historial de pagos; si intenta ver el de otro, recibe un error 403.	MembershipController, PaymentController, PaymentService
RF-08	El sistema debe permitir registrar la entrada y la salida de un cliente al gimnasio usando su código de acceso, rechazando entradas duplicadas y salidas sin una entrada previa.	Si el código es válido, se registra la entrada (error 201). Si se intenta registrar una segunda entrada sin haber salido, el sistema responde con un error 409. Si se intenta registrar una salida sin haber entrado, también responde con un error 409. Si el código no existe, responde con un error 404.	AttendanceController, AttendanceService
RF-09	El administrador debe poder crear entrenadores (con su usuario asociado), y los entrenadores deben poder crear rutinas con ejercicios (nombre, series, repeticiones y orden) y asignarlas a los clientes que tienen a cargo.	Un entrenador puede crear rutinas y ejercicios; un administrador no puede crear rutinas (error 403). Si un cliente no tiene rutina asignada, la consulta devuelve un error 404. Los ejercicios de una rutina se listan en el orden definido.	TrainerController, RoutineController, ExerciseController

RF-10	Un administrador o un recepcionista debe poder consultar un resumen general del gimnasio (clientes activos, ingresos del mes, asistencias del mes y plan más vendido) y una lista de los pagos más recientes con un límite configurable.	El resumen devuelve los totales calculados. La lista de pagos recientes respeta el parámetro de límite (por defecto 5). Un usuario sin rol de administrador o recepcionista recibe un error 403.	ReportController, ReportService

Tabla 1. Requisitos funcionales (RF)

Requisitos no funcionales (RNF)

Los requisitos no funcionales son los que nos dicen qué tan bien debe comportarse el sistema. Nos enfocamos en tres: el rendimiento, la seguridad y la mantenibilidad. Los tres son medibles, y el de mantenibilidad es el que nos conecta directamente con las métricas que aplicamos más adelante.

ID	Categoría	Descripción medible	Forma de verificación
RNF-01	Rendimiento	Las consultas de listado de clientes, planes y membresías deben responder en dos segundos o menos en el 95% de los casos, incluso con hasta cien registros por página.	Se hacen pruebas de carga con curl o JMeter midiendo el tiempo de respuesta de GET /members, GET /plans y GET /memberships con cien registros, y se registra el percentil 95.
RNF-02	Seguridad	Las contraseñas deben guardarse cifradas con BCrypt y todos los endpoints protegidos deben exigir un token JWT válido, con firma verificable y fecha de expiración.	Se revisa el código de SecurityConfig y del codificador de contraseñas, y se corren las pruebas automatizadas que verifican que sin token se responde con 401 y que con un rol insuficiente se responde con 403.
RNF-03	Mantenibilidad	Ningún método del código fuente debe superar una complejidad ciclomática V(G) de 10.	Se hace análisis estático con Lizard sobre la carpeta src/main/java, se genera un reporte con la V(G) de cada método y se identifican los que superan el umbral.

Tabla 2. Requisitos no funcionales (RNF)

ARQUITECTURA

Cuando tuvimos claros los requisitos, el siguiente paso fue decidir cómo íbamos a organizar el código. No queríamos un montón de archivos sueltos sin orden, ni tampoco irnos a lo más complejo solo porque suena bien. Buscábamos un punto medio: una estructura que nos permitiera trabajar en equipo sin pisarnos, que fuera fácil de entender y, sobre todo, que nos facilitara medir la calidad después.

Estilo seleccionado

Elegimos un monolito modular con arquitectura en capas. Suena técnico, pero la idea es sencilla: el sistema es una sola aplicación (un monolito), pero por dentro está dividido en módulos independientes que se encargan cada uno de una parte del negocio. Los módulos que tenemos son: identity (usuarios, roles y seguridad), members (clientes), plans (planes de membresía), billing (membresías y pagos), attendance (asistencia), training (entrenadores, rutinas y ejercicios), reports (reportes) y platform (configuración transversal, como seguridad, CORS y manejo de errores).

Cada módulo, por dentro, sigue las mismas capas: el controlador recibe las peticiones HTTP, los DTO transportan los datos, la entidad representa la tabla de la base de datos, el repositorio se encarga del acceso a datos y el servicio contiene la lógica de negocio. Además, cada módulo expone una Fachada (Facade), que es la única puerta de entrada para los demás módulos. Es decir, un módulo nunca toca directamente las entidades o los repositorios de otro; siempre pasa por su Fachada.

Justificación

Elegimos este estilo por varias razones concretas.

Primero, porque el despliegue es mucho más simple que con microservicios. Todo el backend es un solo archivo JAR y un solo contenedor. Para un proyecto académico y para un gimnasio pequeño o mediano, montar y mantener varios servicios independientes habría sido un esfuerzo que no se justificaba.

Segundo, porque las llamadas entre módulos son en memoria, no por red. Eso ayuda directamente al requisito de rendimiento (RNF-01), ya que no hay latencia de red entre una parte y otra del sistema.

Tercero, porque las operaciones que cruzan módulos, como registrar un cliente y al mismo tiempo crear su membresía y su pago inicial, se resuelven en una sola transacción de base de datos. Con microservicios eso habría requerido patrones más complejos, como sagas o compensaciones, que se salían del alcance.

Cuarto, porque el bajo acoplamiento y la alta cohesión son más fáciles de lograr. Cada módulo agrupa todo lo que tiene que ver con su parte del negocio, y solo se comunica con los demás a través de su Fachada. Eso también nos facilita la medición: podemos analizar la complejidad, el acoplamiento y la cohesión por módulo y ver cuál está más cargado.

Y quinto, porque es una arquitectura que se entiende rápido. Cualquiera del equipo puede abrir el proyecto, ver las carpetas por módulo y saber dónde está cada cosa. Eso ayuda al trabajo en equipo y a la hora de defender el proyecto.

Ahora, también somos conscientes de los compromisos que aceptamos. No podemos escalar un módulo de forma independiente, como sí se haría con microservicios; si el gimnasio creciera mucho, tendríamos que escalar toda la aplicación. Además, todo el backend está en un solo lenguaje y framework, Java con Spring Boot, lo que limita la poliglotía pero también simplifica el mantenimiento. Y existe el riesgo de que, si no se respetan las fronteras entre módulos, el monolito termine convertido en un "espagueti". Para evitarlo, dejamos como regla que ningún módulo acceda directamente a las entidades o repositorios de otro, sino solo a su Fachada. Esa regla se revisa en cada pull request.

Diagramas

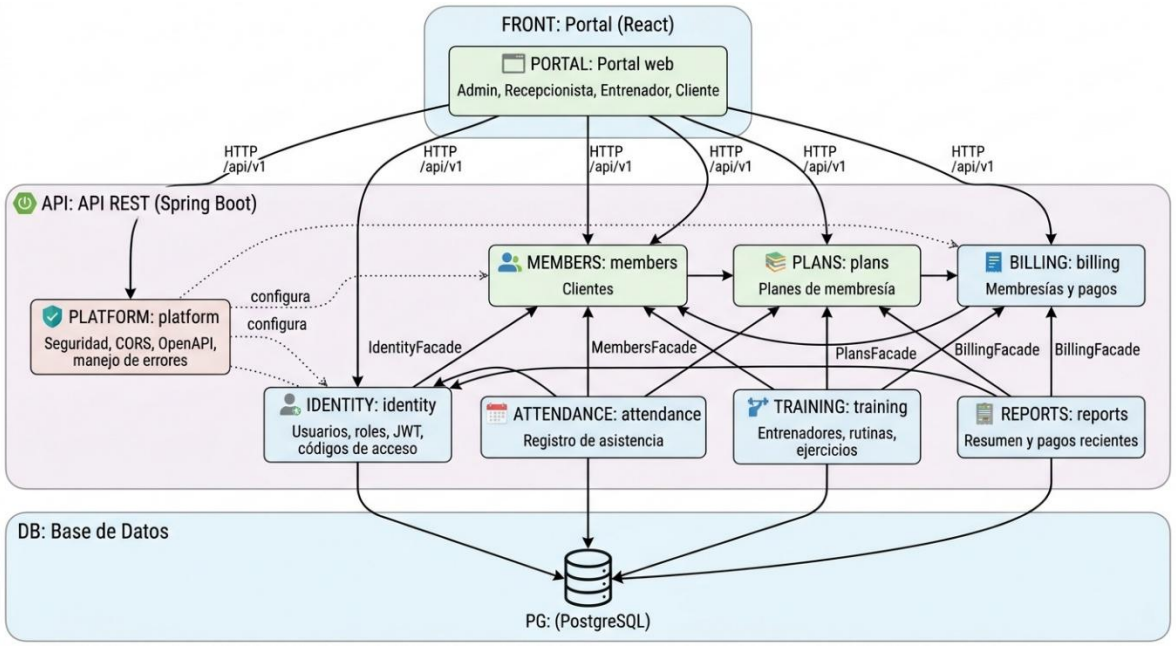


Figura 1. Diagrama de componentes.

El diagrama de componentes muestra cómo el portal (React) se comunica con la API por HTTP. Dentro de la API están los módulos de negocio, y cada uno se conecta con los demás solo a través de su Fachada. Por ejemplo, billing consulta a members y a plans por sus Fachadas, y nunca toca sus tablas directamente. platform es transversal: configura la seguridad, el CORS, la documentación de OpenAPI y el manejo global de errores para todos los módulos. Todos los módulos comparten la misma base de datos PostgreSQL.

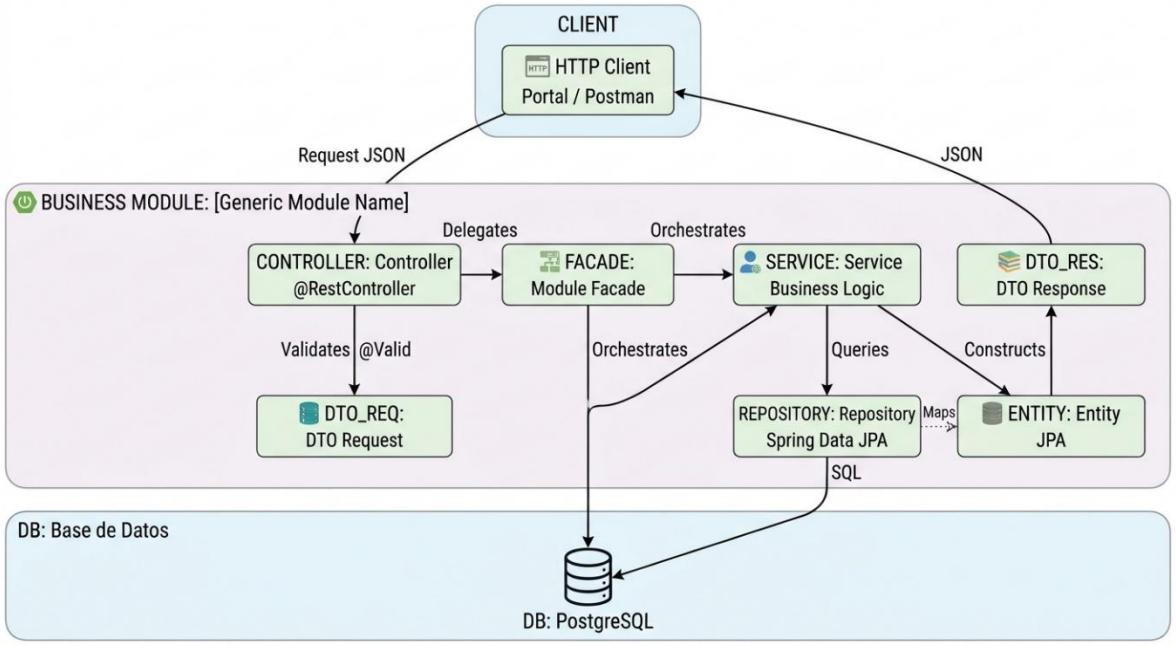


Figura 2. Diagrama de capas.

El diagrama de capas muestra el recorrido de una petición dentro de un módulo. El cliente HTTP envía la petición, el controlador la recibe y valida la entrada, luego pasa al Facade del módulo, que orquesta el servicio. El servicio aplica la lógica de negocio y consulta al repositorio, que finalmente accede a la base de datos. La respuesta vuelve en forma de DTO hasta el cliente. Esta separación es la que nos permite, más adelante, ver que la complejidad se concentra en los servicios y que los controladores y DTO son simples, con baja complejidad.

Estructura del proyecto

La estructura de carpetas refleja esta arquitectura. En la raíz del repositorio tenemos el README.md, la carpeta docs con toda la documentación del proyecto, la carpeta infra con los archivos de Docker, la carpeta metricas con los scripts y los resultados de las mediciones, la carpeta src con el código fuente y la carpeta tests con pruebas adicionales, como la de carga para el requisito de rendimiento.

Dentro de src están las tres partes del sistema: api (el backend en Spring Boot), portal (el frontend en React) y db (las migraciones de la base de datos con Flyway). En src/api/src/main/java/com/gms están los módulos que mencionamos antes, cada uno con sus capas internas. Por ejemplo, el módulo members tiene su

controller, sus dto, su entity, su repository y su service, y dentro de service está la MembersFacade.

En el frontend, dentro de src/portal/src, tenemos las carpetas api, auth, components, pages, test, types y utils. Las páginas están organizadas por funcionalidad y por rol: Dashboard, Members, Plans, Payments, Attendance, Kiosk, Users, Roles, Trainers, Routines, AssignedMembers, MyProfile, MyPayments y MyRoutine.

En src/db están las migraciones versionadas con Flyway, desde la V001 hasta la V010, y la carpeta seed con datos de prueba. En metricas tenemos tres cosas: el README.md que explica cómo se generaron los resultados, la carpeta scripts con los scripts de PowerShell que ejecutan Lizard y CK, y la carpeta resultados con los archivos CSV que salieron de esas mediciones, incluyendo una subcarpeta post con los resultados de después de la refactorización. En tests está el README.md que explica qué pruebas adicionales hay y el script carga-rnf01.ps1, que sirve para medir el tiempo de respuesta de los listados y verificar el requisito de rendimiento.

Esta separación por módulos, por capas y por tipo de archivo es la que nos permite medir por separado, reproducir los resultados y entender en qué parte del código están los problemas.

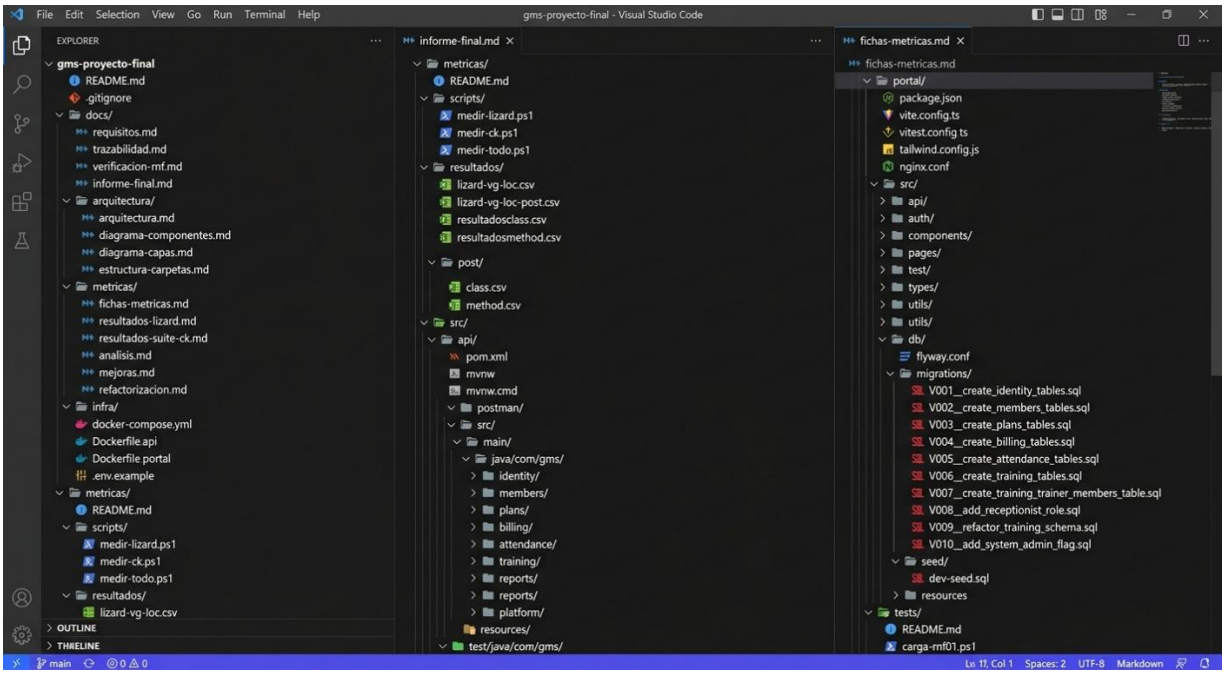


Figura 3. Estructura de carpetas del repositorio, organizada por módulos y capas.

IMPLEMENTACIÓN

Una vez definidos los requisitos y la arquitectura, nos pusimos a construir y a organizar el código. Esta sección cuenta con qué tecnologías lo hicimos, cómo quedaron los módulos, cómo se ejecuta el proyecto, qué pruebas tenemos y cómo se relaciona cada requisito con su parte del código y con su prueba.

Lenguaje y tecnologías

El backend está hecho en Java 21 con Spring Boot, usando Spring Web para los endpoints, Spring Data JPA para el acceso a la base de datos y Spring Security para la autenticación. La seguridad se maneja con JWT, y la documentación de la API se genera con OpenAPI y Swagger. El proyecto se compila con Maven, y trae su propio wrapper (mvnw y mvnw.cmd) para no depender de una instalación aparte.

El frontend está hecho en React con TypeScript, usando Vite como herramienta de construcción y servidor de desarrollo, Tailwind CSS para los estilos y Vitest para las pruebas. En producción, el frontend se sirve con Nginx dentro del contenedor.

La base de datos es PostgreSQL 16, y las migraciones se manejan con Flyway, versionadas desde la V001 hasta la V010. Todo el sistema se levanta con Docker Compose, que orquesta la base de datos, la API y el portal.

Descripción de los módulos

El backend está organizado en módulos, y cada uno se encarga de una parte del negocio. La idea es que cada módulo tenga todo lo suyo adentro y que se comunique con los demás solo a través de su Fachada.

identity es el módulo de usuarios, roles, autenticación con JWT y códigos de acceso. Es el que maneja el login, la creación de usuarios, la activación y desactivación, y la generación de los códigos que se usan en el kiosco.

members es el módulo de clientes. Se encarga del registro, la consulta, la actualización, el cambio de plan, la asignación de entrenador y la activación o desactivación del cliente.

plans es el módulo de planes de membresía. Permite crear, consultar, actualizar, activar y desactivar planes, cada uno con su nombre, precio y duración en días.

billing es el módulo de membresías y pagos. Aquí se crean las membresías, se registran los pagos y se valida que un cliente no tenga dos membresías activas al mismo tiempo y que los pagos no superen el saldo de la membresía.

attendance es el módulo de asistencia. Registra la entrada y la salida de un cliente usando su código de acceso, y rechaza entradas duplicadas o salidas sin entrada previa.

training es el módulo de entrenadores, rutinas y ejercicios. Permite crear entrenadores, crear rutinas con sus ejercicios y asignar rutinas a los clientes que tiene a cargo cada entrenador.

reports es el módulo de reportes. Es de solo lectura y consulta a los demás módulos para armar el resumen general del gimnasio y la lista de pagos recientes.

platform es el módulo transversal. No tiene lógica de negocio, sino la configuración de seguridad, CORS, OpenAPI y el manejo global de errores. También tiene el AdminSeeder, que crea el usuario administrador del sistema al arrancar.

En el frontend, las páginas están organizadas por funcionalidad y por rol. Están Login, Dashboard, Members y MemberDetail, Plans, Payments, Attendance y Kiosk, Users y Roles, Trainers, Routines y AssignedMembers, y las vistas del propio cliente: MyProfile, MyPayments y MyRoutine.

Instrucciones de ejecución

El proyecto se ejecuta con Docker Compose. Los pasos son los siguientes.

Primero, se clona el repositorio y se entra a la carpeta del proyecto. Luego, dentro de la carpeta infra, se copia el archivo .env.example a .env. Después, se ejecuta docker compose up --build y se espera a que levanten los tres contenedores: la base de datos, la API y el portal. Cuando todo está arriba, el portal queda disponible en http://localhost:5173, Swagger en http://localhost:8080/api/v1/swagger-ui.html, y el login inicial es admin con contraseña admin123.

Para correr las pruebas del backend, se levanta solo la base de datos con docker compose up -d db, se entra a src/api y se ejecuta .\mvnw.cmd test. El resultado esperado es que pasen las 114 pruebas y el build sea exitoso. Para las pruebas del frontend, se entra a src/portal, se ejecuta npm install y luego npm run test. El resultado esperado es que pasen las 2 pruebas.

Para volver a medir las métricas, se pueden usar los scripts que dejamos en metricas/scripts. El script medir-lizard.ps1 ejecuta Lizard y guarda el CSV de V(G) y LOC. El script medir-ck.ps1 ejecuta CK y guarda los CSV de la suite CK. Y el script medir-todo.ps1 ejecuta los dos anteriores de una vez.

Pruebas

El backend tiene 114 pruebas hechas con JUnit y MockMvc, organizadas por módulo. Las pruebas cubren los controladores de cada módulo, el servicio de pagos, las reglas de roles y permisos, y el módulo de reportes. Todas las pruebas levantan el contexto de Spring y necesitan la base de datos en ejecución.

El frontend tiene 2 pruebas hechas con Vitest, que cubren el componente MetricCard y la configuración de pruebas.

Además de las pruebas unitarias e integración, en la carpeta tests dejamos un script de prueba de carga (carga-rnf01.ps1) para verificar el requisito de rendimiento. Ese script mide el tiempo de respuesta de los listados de clientes, planes y membresías, y calcula el percentil 95.

Tabla de trazabilidad

La tabla de trazabilidad relaciona cada requisito funcional con el módulo y la clase que lo implementa, y con las pruebas que lo respaldan. La tabla completa está en docs/trazabilidad.md, y aquí la resumimos.

RF	Módulo	Componente	Pruebas principales
RF-01	identity	AuthController, AuthService, JwtService	AuthControllerTest.login_whenValidCredentials_returnsToken(), login_whenWrongPassword_returns401(), login_whenUsernameDoesNotExist_returns401()
RF-02	identity	UserController, UserService	UserControllerTest.createUser_whenValidRequest_returns201WithUser(), UserManagementControllerTest.listUsers_asAdmin_returns200(), listUsers_asReceptionist_returns403(), deactivateSystemAdmin_returns409()

RF-03	members	MemberController, MemberService	MemberControllerTest.registerMember_whenValidRequest_returns201(), registerMember_whenDuplicateDocumentNumber_returns409(), registerMember_whenDuplicateUsername_returns409(), registerMember_withMemberRole_returns403()
RF-04	members	MemberController, MemberService	ListMembersControllerTest.listMembers_whenAuthenticated_returnsPage(), getMember_whenNotFound_returns404(), UpdateMemberControllerTest.updateMember_whenFullNameAndContactProvided_updatesBoth(), ActivateMemberControllerTest.activateMember_whenInactive_returns204()
RF-05	plans	PlanController, PlanService	CreatePlanControllerTest.createPlan_whenValidRequest_returns201(), createPlan_whenDuplicateName_returns409(), ListPlansControllerTest.listPlans_whenAuthenticated_returnsPage(), UpdatePlanControllerTest.updatePlan_whenNameCollidesWithAnotherActive_returns409(), DeactivatePlanControllerTest.deactivatePlan_whenActive_returns204()
RF-06	billing	MembershipController, MembershipService	CreateMembershipControllerTest.createMembership_whenValid_returns201(), createMembership_whenMemberHasActiveMembership_returns409(), createMembership_whenPlanInactive_returns409(), ListMembershipsControllerTest.listMemberships_whenAuthenticated_returnsPage()

RF-07	billing	MembershipController, PaymentController, PaymentService	RecordPaymentControllerTest.recordPayment_whenValid_returns201(), recordPayment_whenPartialThenExceeds_returns409(), PaymentServiceTest.record_whenValidPayment_savesAndReturnsPayment(), record_whenPaymentExceedsLockedPrice_throwsPaymentExceedsBalanceException()
RF-08	attendance	AttendanceController, AttendanceService	AttendanceControllerTest.registerEntry_whenValid_returns201AndCreatesRecord(), registerEntry_whenAlreadyCheckedIn_returns409(), registerExit_whenCheckedIn_returns201(), registerExit_whenNotCheckedIn_returns409(), registerEntry_whenInvalidCode_returns404()
RF-09	training	TrainerController, RoutineController, ExerciseController	TrainerControllerTest.createTrainer_asAdmin_returns201(), RoutineControllerTest.createRoutine_asTrainer_returns201(), createRoutine_asAdmin_returns403(), ExerciseControllerTest.createExercise_asTrainer_returns201(), createExercise_withInvalidSets_returns400()
RF-10	reports	ReportController, ReportService	ReportControllerTest.summary_asAdmin_returnsTotals(), summary_asMember_returns403(), recentPayments_asAdmin_returnsList(), recentPayments_whenLimitProvided_respectsLimit(), recentPayments_withoutToken_returns401()

Tabla 3. Trazabilidad

Con esta tabla cerramos la parte de implementación. Cada requisito tiene al menos una prueba que lo respalda, y todos los componentes están ubicados en su módulo correspondiente dentro de src/api/src/main/java/com/gms.

DEFINICIÓN DE MÉTRICAS

Después de tener el sistema funcionando y organizado, llegó el momento de medirlo. Para eso escogimos tres métricas de las que vimos en el curso, siguiendo la regla de que al menos una tiene que ser estructural (complejidad ciclomática o líneas de código) y al menos una tiene que evaluar el diseño o la evolución (suite CK, índice de mantenibilidad o costo de mantenimiento). Las tres las aplicamos sobre el backend en Java, que es la parte del sistema donde está la lógica de negocio y donde tiene más sentido medir la mantenibilidad.

La primera es la complejidad ciclomática V(G) de McCabe, que cuenta los caminos de ejecución que puede tomar un método. La elegimos porque es la métrica estructural más directa para saber si un método es fácil o difícil de entender y de probar, y porque conecta directamente con nuestro requisito no funcional de mantenibilidad (RNF-03), que dice que ningún método debe superar una V(G) de 10.

La segunda es el conteo de líneas de código (LOC), específicamente las líneas efectivas, que son las que no son comentarios ni líneas en blanco. La elegimos porque complementa a la complejidad: un método puede tener baja complejidad pero ser muy largo, y eso también es una señal de alerta. Además, el porcentaje de comentarios nos dice si el código está documentado o no.

La tercera es la suite CK de Chidamber y Kemerer, que mide la calidad del diseño orientado a objetos a través de seis indicadores: WMC (complejidad ponderada de la clase), DIT (profundidad de herencia), NOC (número de hijos), CBO (acoplamiento entre objetos), RFC (respuesta de la clase) y LCOM (falta de cohesión de los métodos). La elegimos porque es la métrica de diseño que nos permite detectar problemas como clases con demasiadas responsabilidades (God Class), alto acoplamiento o baja cohesión, que son justamente los problemas que más afectan la mantenibilidad a largo plazo.

Con esas tres métricas cubrimos lo estructural y lo de diseño, y podemos cruzar los resultados para confirmar los problemas. Por ejemplo, si una clase tiene un WMC alto y además contiene el único método con V(G) mayor a 10, ahí hay una señal clara de que algo hay que refactorizar.

A continuación, presentamos las fichas de cada métrica, con lo que mide, cómo se calcula, el umbral de referencia, la herramienta que usamos y sobre qué parte del código la aplicamos.

Ficha 1: Complejidad Ciclomática V(G) (McCabe)

Campo	Contenido
Nombre de la métrica	Complejidad Ciclomática V(G) (McCabe)
Atributo que mide	La complejidad de control de flujo de un método, es decir, el número de caminos linealmente independientes que puede tomar.
Fórmula o forma de cálculo	En la práctica, V(G) = número de puntos de decisión + 1. Formalmente, V(G) = E - N + 2P, donde E son las aristas, N los nodos y P los componentes conectados.
Umbral de referencia e interpretación	De 1 a 10: riesgo bajo. De 11 a 20: riesgo moderado. De 21 a 50: riesgo alto. Más de 50: riesgo muy alto.
Herramienta o script de medición	Lizard versión 1.25.0. El comando está en metricas/scripts/medir-lizard.ps1.
Alcance de la medición	Todos los métodos de src/api/src/main/java/com/gms/, en los módulos identity, members, plans, billing, attendance, training, reports y platform.
Justificación	Es una métrica estructural y conecta directamente con el RNF-03. Permite identificar los métodos que necesitan refactorización con Extract Method.
Acción asociada si se supera el umbral	Aplicar Extract Method para dividir el método en partes más pequeñas y con una sola responsabilidad.

Ficha 2: Líneas de código (LOC) y porcentaje de comentarios

Campo	Contenido
Nombre de la métrica	Líneas de código efectivas (NLOC) y porcentaje de comentarios
Atributo que mide	El tamaño del código, sin contar comentarios ni líneas en blanco, y la proporción de documentación.
Fórmula o forma de cálculo	NLOC son las líneas efectivas de código. El porcentaje de comentarios se calcula como (líneas de comentario / líneas totales) × 100.
Umbral de referencia e interpretación	Más de 50 líneas efectivas por método es una señal de alerta. Un porcentaje de comentarios muy bajo (menos del 10%) o muy alto (más del 50%) puede indicar problemas de documentación.
Herramienta o script de medición	Lizard versión 1.25.0. El comando está en metricas/scripts/medir-lizard.ps1.
Alcance de la medición	Todos los métodos de src/api/src/main/java/com/gms/.

Justificación	Es una métrica estructural que complementa a la complejidad. Un método puede tener baja V(G) pero ser demasiado largo, y eso también es un problema de mantenibilidad.
Acción asociada si se supera el umbral	Aplicar Extract Method para reducir el tamaño del método. Si el porcentaje de comentarios es bajo, agregar documentación con Javadoc.

Ficha 3: Suite CK (WMC, DIT, NOC, CBO, RFC, LCOM)

Campo	Contenido
Nombre de la métrica	Suite CK de Chidamber y Kemerer
Atributo que mide	La calidad del diseño orientado a objetos a través de seis indicadores: WMC (complejidad ponderada de la clase), DIT (profundidad de herencia), NOC (número de hijos), CBO (acoplamiento entre objetos), RFC (respuesta de la clase) y LCOM (falta de cohesión de los métodos).
Fórmula o forma de cálculo	WMC es la suma de las complejidades de los métodos de la clase. DIT es la profundidad en el árbol de herencia. NOC es el número de subclases directas. CBO es el número de clases con las que está acoplada. RFC es el número de métodos que pueden ejecutarse en respuesta a un mensaje. LCOM mide la falta de cohesión entre los métodos.
Umbral de referencia e interpretación	WMC mayor a 20 indica una clase con demasiadas responsabilidades. DIT mayor a 5 indica herencia profunda. CBO mayor a 10 indica alto acoplamiento. RFC mayor a 50 indica una respuesta excesiva. LCOM mayor a 0.7 indica baja cohesión.
Herramienta o script de medición	CK de Maurício Aniche, versión 0.7.0. El comando está en metricas/scripts/medir-ck.ps1.
Alcance de la medición	Todas las clases de src/api/src/main/java/com/gms/. Se midieron 94 clases en total.
Justificación	Es la métrica de diseño y evolución que exige la regla de selección. Permite identificar clases con problemas de diseño como God Class, alto acoplamiento o baja cohesión, que son los que más afectan la mantenibilidad.
Acción asociada si se supera el umbral	Para WMC alto, aplicar Extract Class. Para CBO alto, aplicar inyección de dependencias o una Fachada. Para LCOM alto, aplicar Extract Class o Move Method. Para RFC alto, aplicar Extract Method o Extract Class.

RESULTADOS DE LAS MÉTRICAS

Una vez aplicadas las tres métricas, organizamos los resultados en tablas para poder compararlos con los umbrales de referencia. En total medimos 201 métodos y 94 clases del backend. Los resultados completos están en metricas/resultados/, y aquí presentamos los más importantes.

Complejidad Ciclomática V(G)

De los 201 métodos medidos, 200 tienen una V(G) de 10 o menos, lo que representa el 99.5% del código. Solo un método supera el umbral: MemberService.updateFullProfile(), con una V(G) de 15. Ese método está en riesgo moderado según la tabla de referencia, y es justamente el que refactorizamos más adelante.

Rango V(G)	Métodos	Porcentaje
1 a 10 (riesgo bajo)	200	99.5%
11 a 20 (riesgo moderado)	1	0.5%
21 a 50 (riesgo alto)	0	0%
Más de 50 (riesgo muy alto)	0	0%

Tabla 4. Complejidad Ciclomática V(G)

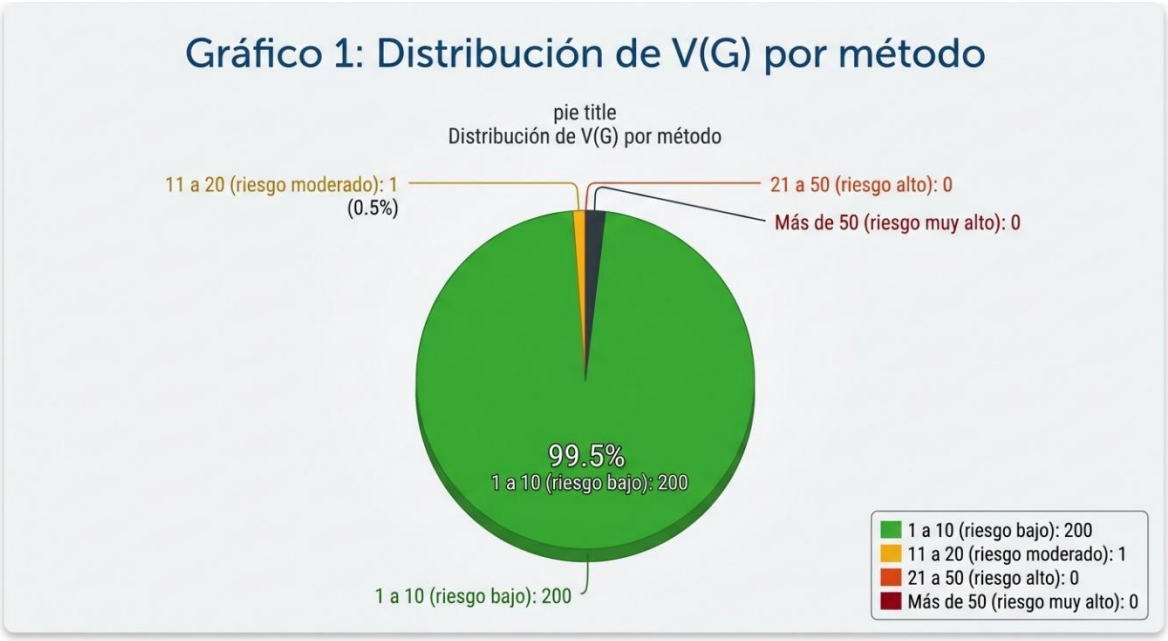


Figura 4. Distribución de la complejidad ciclomática V(G) por método.

Se observa que la gran mayoría de los métodos se concentra en el rango de riesgo bajo (1 a 10).

El único método que supera el umbral es el siguiente.

Método	V(G)	NLOC	Umbral	Estado
MemberService.updateFullProfile()	15	47	≤ 10	Riesgo moderado. Requiere Extract Method.

Tabla 5. Método que supera el umbral V(G)

Líneas de código (LOC)

De los 201 métodos medidos, 200 tienen 50 líneas efectivas o menos, lo que también representa el 99.5% del código. Solo un método supera el umbral: SecurityConfig.securityFilterChain(), con 67 líneas efectivas. Ese método tiene una V(G) de 1, así que no es un problema de complejidad sino de tamaño: son muchas reglas de seguridad encadenadas en un solo método.

Método	NLOC	V(G)	Umbral	Estado
SecurityConfig.securityFilterChain()	67	1	≤ 50	Señal de alerta por tamaño. Se puede dividir por legibilidad, pero no afecta la complejidad.

Tabla 6. Método que supera el umbral LOC

Suite CK

En la suite CK medimos 94 clases. Los resultados más relevantes son los siguientes.

WMC mayor a 20 (clases con demasiadas responsabilidades):

Clase	WMC	CBO	RFC	Estado
MemberService	39	24	54	God Class. Supera el umbral de WMC, CBO y RFC.
UserService	23	13	26	Supera el umbral de WMC.
GlobalExceptionHandler	21	33	6	Supera el umbral de WMC y de CBO.

Tabla 7. WMC mayor a 20

CBO mayor a 10 (clases con alto acoplamiento):

Se encontraron 22 clases que superan el umbral de CBO. Las más críticas son GlobalExceptionHandler (CBO 33), MemberService (CBO 24), UserController (CBO 22) y MemberController (CBO 22). Muchos de estos acoplamientos son esperables en un monolito modular, porque los controladores inyectan Fachadas y

servicios, y los servicios inyectan repositorios. Por eso no todos se consideran un problema real, sino solo los que se combinan con un WMC alto o un RFC alto.

RFC mayor a 50 (clases con respuesta excesiva):

Clase	RFC	WMC	Estado
MemberService	54	39	Confirma que es una God Class.

Tabla 8. RFC mayor a 50

LCOM mayor a 0.7 (clases con baja cohesión):

Se encontraron 12 clases que superan el umbral de LCOM, pero todas son entidades JPA (Member, User, Membership, Plan, Payment, Exercise, Routine, AccessCode, Trainer, TrainerMember, RoutineAssignment y AttendanceRecord). En estas clases, los métodos de acceso (getters y setters) no comparten estado entre sí, lo que infla el LCOM. Por eso las consideramos falsos positivos y no proponemos refactorización para ellas.

DIT mayor a 5 (herencia profunda):

Se encontraron 6 clases que superan el umbral de DIT, pero todas son servicios que extienden clases de Spring. La profundidad de herencia real del dominio es 1, porque nuestras clases no heredan de otras clases nuestras. Por eso también lo consideramos ruido de la herramienta y no un problema de diseño.

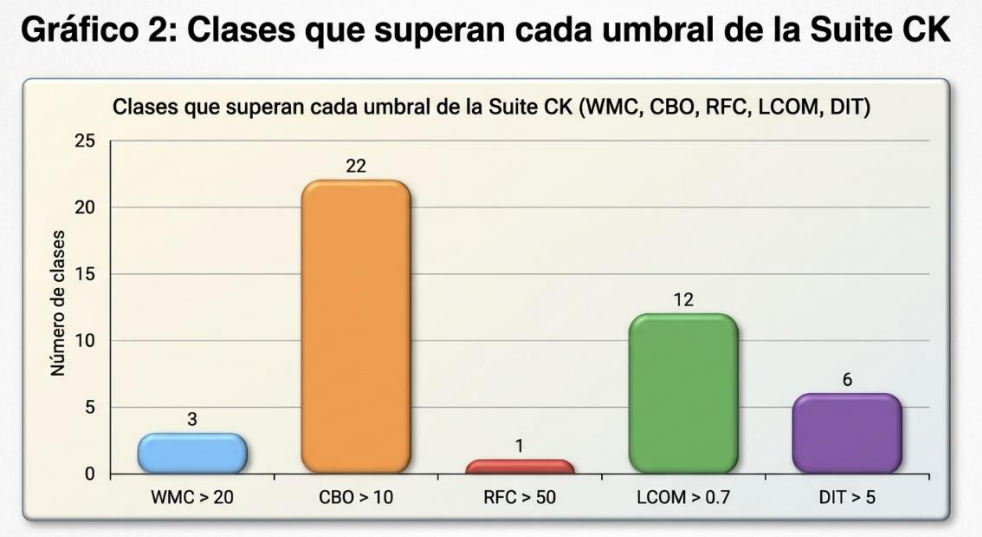


Figura 5. Clases que superan cada umbral de la Suite CK.

Se observa que el CBO es el umbral que más clases supera (22), seguido de LCOM (12) y DIT (6). Sin embargo, los de LCOM y DIT son falsos positivos

(entidades JPA y herencia de Spring). Los umbrales que reflejan problemas reales son WMC (3 clases) y RFC (1 clase).

Resumen general

Métrica	Umbral	Clases o métodos que lo superan	Observación
V(G)	≤ 10	1 método (updateFullProfile)	Único método en riesgo moderado.
NLOC	≤ 50	1 método (securityFilterChain)	Señal de alerta por tamaño, no por complejidad.
WMC	≤ 20	3 clases (MemberService, UserService, GlobalExceptionHandler)	MemberService es la más crítica.
CBO	≤ 10	22 clases	Muchos son esperables en un monolito modular.
RFC	≤ 50	1 clase (MemberService)	Confirma que es una God Class.
LCOM	≤ 0.7	12 clases (todas entidades JPA)	Falsos positivos.
DIT	≤ 5	6 clases (todas servicios de Spring)	Ruido de la herramienta.

Tabla 9. Resumen general

Los resultados muestran que el código tiene una calidad general buena: casi todos los métodos cumplen con la complejidad y el tamaño esperados. El problema principal está concentrado en MemberService, que es la clase más compleja y la que más responsabilidades tiene. Esa es la que vamos a analizar y refactorizar en la siguiente sección.

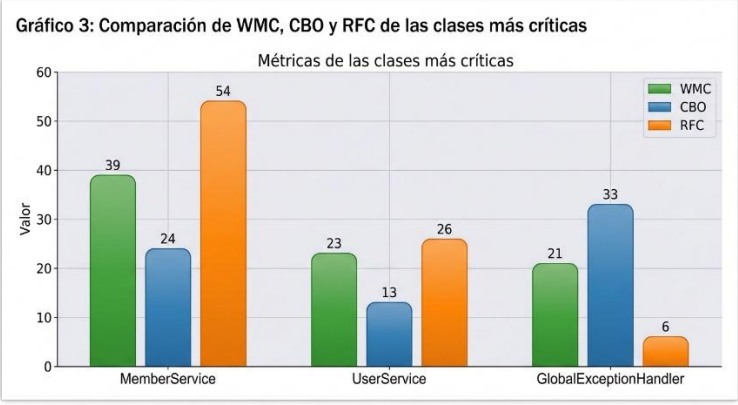


Figura 6. Comparación de WMC, CBO y RFC de las tres clases más críticas.

MemberService es la que tiene los valores más altos en las tres métricas (WMC 39, CBO 24, RFC 54), lo que confirma que es una God Class. GlobalExceptionHandler tiene un CBO muy alto (33) pero un RFC bajo (6), lo que indica alto acoplamiento sin exceso de respuesta. UserService tiene valores moderados en las tres métricas.

ANÁLISIS Y PLAN DE MEJORAS

Con los resultados de las métricas sobre la mesa, el siguiente paso fue interpretarlos con calma y decidir qué valía la pena cambiar. No se trataba de refactorizar por refactorizar, sino de identificar los problemas reales, priorizarlos según el riesgo y el esfuerzo, y proponer mejoras con una técnica concreta detrás. Además, como el profesor recomendaba implementar al menos una refactorización y volver a medir, elegimos la más crítica y la llevamos a cabo para demostrar el ciclo completo.

Interpretación frente a los umbrales

En general, el código tiene una calidad buena. La complejidad ciclomática y las líneas de código están bajo control en casi todo el proyecto: de 201 métodos medidos, 200 cumplen con V(G) ≤ 10 y 200 cumplen con NLOC ≤ 50. Eso quiere decir que la arquitectura en capas y la separación por módulos están funcionando, porque la lógica está repartida en métodos pequeños y con una sola responsabilidad.

El problema no está en la mayoría del código, sino en unos pocos puntos concentrados. En la suite CK aparecen tres clases que superan el umbral de WMC mayor a 20: MemberService con 39, UserService con 23 y GlobalExceptionHandler con 21. De esas tres, MemberService es la más crítica, porque además supera el umbral de RFC mayor a 50 con 54, y tiene el único método del proyecto con V(G) mayor a 10 (updateFullProfile, con 15). Es decir, las tres métricas apuntan al mismo lugar, y eso no es casualidad: es una señal de que ahí hay un problema de diseño real.

Hay otros resultados que también superan umbrales, pero que interpretamos con cuidado. Por ejemplo, 22 clases superan el CBO mayor a 10. Muchas de ellas son controladores y Fachadas que inyectan dependencias, lo cual es normal en un monolito modular; no lo consideramos un problema grave, sino un acoplamiento esperado. También hay 12 clases con LCOM mayor a 0.7, pero todas son entidades JPA, donde los getters y setters no comparten estado entre sí y por eso el LCOM se infla. Y hay 6 clases con DIT mayor a 5, pero todas son servicios que

extienden clases de Spring, así que la profundidad de herencia real del dominio es uno. A esos dos últimos los tratamos como falsos positivos y no les propusimos refactorización.

Problemas detectados

Después de interpretar los resultados, identificamos cuatro problemas y los priorizamos según el riesgo y el esfuerzo.

Prioridad	Problema	Métricas que lo evidencian	Riesgo	Esfuerzo
Alta	MemberService es una God Class, con demasiadas responsabilidades y un método con V(G) de 15.	WMC 39, RFC 54, CBO 24, V(G) 15 en updateFullProfile.	Alto	Medio
Media	GlobalExceptionHandler maneja las excepciones de todos los módulos en una sola clase.	WMC 21, CBO 33.	Medio	Bajo
Media	UserService concentra demasiadas operaciones de usuario.	WMC 23, CBO 13.	Medio	Bajo
Baja	SecurityConfig.securityFilterChain() tiene 67 líneas efectivas en un solo método.	NLOC 67, V(G) 1.	Bajo	Bajo

Tabla 10. Problemas detectados

El problema de MemberService es el más importante porque combina las tres métricas: complejidad alta (WMC), respuesta excesiva (RFC) y el único método del proyecto que supera el umbral de complejidad ciclomática (V(G)). Esa clase concentra el registro de clientes, la actualización de perfil, el cambio de plan, la asignación de entrenador, la activación y desactivación, la regeneración del código de acceso y las búsquedas. Cualquier cambio en la lógica de clientes obliga a tocar esa clase, y eso aumenta el riesgo de regresiones.

Mejoras propuestas

Para cada problema propusimos una técnica de refactorización concreta, explicando qué efecto se espera sobre las métricas.

Para MemberService (prioridad alta): la técnica es Extract Class combinada con Extract Method. La idea es dividir la clase en servicios más especializados, cada uno con una responsabilidad: MemberRegistrationService para el registro, MemberProfileService para la actualización de perfil, MemberPlanService para el

cambio de plan, MemberTrainerService para la asignación de entrenador, MemberActivationService para la activación y desactivación, y MemberQueryService para las búsquedas. Además, dentro de MemberProfileService, aplicar Extract Method para dividir updateFullProfile en pasos pequeños. El efecto esperado es que el WMC, el RFC y el CBO bajen por clase, y que la V(G) de updateFullProfile baje a 4 o menos.

Para GlobalExceptionHandler (prioridad media): la técnica es Extract Class por módulo. Dividir la clase en manejadores especializados: IdentityExceptionHandler, MembersExceptionHandler, BillingExceptionHandler, TrainingExceptionHandler y PlatformExceptionHandler, cada uno anotado con @RestControllerAdvice para que Spring lo aplique solo a su módulo. El efecto esperado es que el CBO y el WMC bajen por clase.

Para UserService (prioridad media): la técnica es Extract Class. Dividir en UserCrudService para crear, listar y actualizar usuarios, y UserActivationService para activar, desactivar, validar el último administrador y proteger al administrador del sistema. El efecto esperado es que el WMC y el CBO bajen.

Para SecurityConfig.securityFilterChain() (prioridad baja): la técnica es Extract Method. Dividir en métodos auxiliares como configurePublicEndpoints(), configureRoleEndpoints() y configureAuthenticatedEndpoints(). El efecto esperado es que el NLOC baje a 25 por método, pero la V(G) no cambia porque no hay ramificaciones.

Refactorización implementada

Aunque la recomendación del PDF decía que implementar una refactorización daba valor adicional, decidimos hacerlo con la más crítica: MemberService.updateFullProfile(). Aplicamos Extract Method, que es la técnica que corresponde cuando el problema es la complejidad de un método.

El método original tenía 47 líneas efectivas y una V(G) de 15, y mezclaba seis responsabilidades: actualizar el nombre, actualizar el contacto, validar y actualizar el documento, actualizar las credenciales, cambiar el plan y asignar el entrenador. Lo dividimos en seis métodos privados (updateFullName, updateContact, updateDocumentNumber, updateCredentials, updatePlan y updateTrainer) y dejamos el método principal solo orquestando las llamadas.

La comparación antes y después quedó así.

Métrica	Antes	Después	Resultado
V(G) de updateFullProfile()	15	≤ 7	Mejoró
Métodos con V(G) mayor a 10 en el proyecto	1	0	Mejoró
V(G) máximo del proyecto	15	7	Mejoró
WMC de MemberService	39	44	Empeoró
RFC de MemberService	54	60	Empeoró
CBO de MemberService	24	24	Igual
LCOM* de MemberService	0.61	0.72	Empeoró
Total de métodos de la clase	14	20	Aumentó por los métodos extraídos

Tabla 11. Comparación antes y después

Lo primero que hay que decir es que la refactorización sí funcionó para lo que queríamos: el único método que superaba el umbral de V(G) quedó por debajo, y el proyecto pasó a tener el 100% de los métodos con V(G) ≤ 10. Eso cumple el RNF-03. Además, las 114 pruebas siguieron pasando, así que el comportamiento de la aplicación se preservó.

Pero también hay que ser honestos con lo que pasó con las otras métricas. El WMC, el RFC y el LCOM de MemberService empeoraron. Esto no es un error de la refactorización, sino una consecuencia esperada de la técnica que elegimos. Extract Method mejora la complejidad por método, pero no reduce las métricas de la clase, porque los métodos extraídos siguen perteneciendo a la misma clase y su complejidad se suma al total. Además, el método principal ahora invoca más métodos, lo que sube el RFC, y los métodos nuevos no comparten estado entre sí, lo que sube el LCOM.

Para mejorar esas métricas de clase, la técnica correcta es Extract Class, que es justamente la que propusimos como siguiente paso en el plan de mejoras. No la implementamos en este proyecto por cuestión de alcance y tiempo, pero queda documentada con su efecto esperado. Y esa es, en el fondo, una de las lecciones más valiosas del trabajo: las métricas no son independientes, y mejorar una puede empeorar otra. Lo importante es entender el trade-off y elegir la técnica que corresponda al problema que se quiere resolver.

Efecto esperado de las mejoras no implementadas

Para que quede claro qué se espera de las mejoras que no se implementaron, dejamos la siguiente proyección.

Problema	Técnica	Efecto esperado
MemberService (God Class)	Extract Class + Extract Method	WMC de 39 a unos 10 por servicio, RFC de 54 a unos 15 por servicio, CBO de 24 a unos 5 por servicio.
GlobalExceptionHandler (CBO 33)	Extract Class por módulo	CBO de 33 a unos 7 por manejador, WMC de 21 a unos 4 por manejador.
UserService (WMC 23)	Extract Class	WMC de 23 a unos 12 por servicio, CBO de 13 a unos 7 por servicio.
SecurityConfig.securityFilterChain() (NLOC 67)	Extract Method	NLOC de 67 a unos 25 por método, V(G) se mantiene en 1.

Tabla 12. Proyección

Con este análisis cerramos la parte de resultados y mejoras. La refactorización implementada demostró el ciclo Medir, Detectar, Refactorizar y Verificar, y las mejoras propuestas quedan documentadas con su técnica y su efecto esperado para futuras iteraciones.

CONCLUSIONES Y LECCIONES APRENDIDAS

Después de recorrer todo el proyecto, desde la definición de los requisitos hasta la refactorización y la re-medición, vale la pena detenerse a mirar qué aprendimos y qué quedó como resultado. Esta sección no es solo un cierre formal, sino una reflexión sobre lo que significó medir nuestro propio software y tomar decisiones con base en datos.

Conclusiones

La primera conclusión es que el código de GMS tiene una calidad general buena. De los 201 métodos que medimos, 200 cumplen con la complejidad ciclomática y con el tamaño esperados. Eso quiere decir que la arquitectura en capas y la separación por módulos funcionaron: la lógica está repartida en métodos

pequeños y con una sola responsabilidad, y no hay un caos de métodos gigantes y enredados. Para un proyecto que nació en otra asignatura y que se adaptó para este curso, ese resultado es satisfactorio.

La segunda conclusión es que el problema no está repartido por todo el código, sino concentrado en unos pocos puntos. La suite CK nos mostró que tres clases superan el umbral de WMC, y de esas tres, MemberService es la más crítica porque además supera el umbral de RFC y contiene el único método del proyecto con V(G) mayor a 10. Es decir, las tres métricas apuntan al mismo lugar, y eso nos da confianza en el diagnóstico: no es una casualidad, es un problema real de diseño.

La tercera conclusión es que la refactorización implementada sirvió para lo que queríamos. Después de aplicar Extract Method en MemberService.updateFullProfile(), el único método que superaba el umbral de V(G) quedó por debajo, y el proyecto pasó a tener el 100% de sus métodos con V(G) ≤ 10. Eso cumple el requisito no funcional de mantenibilidad. Además, las 114 pruebas siguieron pasando, lo que confirma que el comportamiento de la aplicación se preservó. Es decir, mejoramos la calidad interna sin romper lo que el usuario ve.

La cuarta conclusión es que las métricas de clase no mejoraron con esa refactorización, y eso es importante decirlo con honestidad. El WMC, el RFC y el LCOM de MemberService empeoraron, no porque la refactorización estuviera mal hecha, sino porque Extract Method no es la técnica que corresponde para mejorar esas métricas. Para eso se necesita Extract Class, que quedó propuesta en el plan de mejoras, con su técnica y su efecto esperado, pero que no se implementó por cuestión de alcance y tiempo. Y esa es una de las cosas más valiosas que nos llevamos: no todas las refactorizaciones mejoran todas las métricas, y saber qué técnica usar para cada problema es parte del aprendizaje.

Lecciones aprendidas

Las métricas no son independientes. Una de las lecciones más grandes es que las métricas se afectan entre sí. Mejorar la complejidad por método puede empeorar las métricas de clase, y reducir el acoplamiento puede aumentar el número de clases. No se trata de optimizar una sola métrica, sino de entender el trade-off y elegir la técnica que corresponda al problema que se quiere resolver. Esa idea, que en clase suena teórica, la vivimos en carne propia con la refactorización de MemberService.

Las herramientas tienen limitaciones y hay que interpretarlas con criterio. Lizard y CK nos dieron datos muy valiosos, pero también nos mostraron falsos positivos. Las entidades JPA salen con un LCOM alto porque los getters y setters no comparten estado, y los servicios salen con un DIT alto porque heredan de clases de Spring. Si uno se queda con el número sin interpretarlo, podría proponer refactorizaciones que no tienen sentido. Aprender a leer los resultados con contexto fue parte del trabajo.

Medir sin un plan no sirve de mucho. Al principio teníamos los números, pero no sabíamos qué hacer con ellos. Fue cuando definimos los umbrales, comparamos los resultados y priorizamos los problemas que el análisis cobró sentido. La medición no es el final del proceso, es el insumo para tomar decisiones.

La arquitectura facilita o dificulta la medición. El hecho de que GMS sea un monolito modular con capas bien definidas hizo que fuera mucho más fácil medir por módulo y entender en qué parte del código estaban los problemas. Si el código hubiera estado todo mezclado, habríamos tenido los mismos números, pero sin saber de dónde venían. La arquitectura no solo sirve para organizar el código, también sirve para analizarlo.

Documentar con evidencia hace la diferencia. Guardar los CSV de resultados, los scripts de medición y las fichas de cada métrica nos permitió volver atrás, comparar antes y después, y demostrar que los cambios tuvieron el efecto esperado. Sin esa evidencia, las conclusiones habrían sido opiniones. Con la evidencia, son datos.

El trabajo en equipo y el control de versiones son parte de la calidad. No basta con que el código funcione; también importa cómo se organiza el repositorio, cómo se escriben los mensajes de commit, cómo se dividen las ramas por funcionalidad y cómo se revisan los cambios entre compañeros. Esa disciplina, que parece un detalle, es parte de lo que hace que un proyecto sea mantenible a largo plazo.

Reflexión final

Si algo nos llevamos de este proyecto es que la calidad del software no se mide con una sola métrica ni se mejora con una sola refactorización. Se mide con varias métricas que se complementan, se interpreta con criterio y se mejora con la técnica adecuada para cada problema. GMS quedó mejor de lo que estaba, pero sobre todo, nosotros quedamos con una forma más clara de mirar el software: no solo preguntarnos si funciona, sino también preguntarnos cómo está construido por dentro y qué podemos hacer para que sea más fácil de mantener en el futuro.

APORTE INDIVIDUAL

El proyecto fue un trabajo en equipo, y se ve reflejado en el historial de commits del repositorio. Aunque cada uno tuvo roles y responsabilidades distintas, ambos participamos en todas las fases del proyecto, revisando el trabajo del otro y haciendo los merges correspondientes. La imagen de contribuyentes de GitHub muestra una participación equilibrada: Sebastián aportó 7 commits con más de 11,000 líneas agregadas, y Dayana aportó 5 commits con más de 10,000 líneas agregadas, además de ser la encargada de revisar y aprobar los Pull Requests en la mayoría de las fases.

A continuación, describimos el trabajo de cada uno, respaldado por el historial de commits.

Juan Sebastian Agudelo Quintero

Sebastián se enfocó principalmente en el backend y en la parte técnica del proyecto. Su trabajo incluyó:

Migración del backend al monorepositorio: fue el encargado de mover el proyecto de Spring Boot (gms-api) a la nueva estructura de carpetas, ajustando el pom.xml, el wrapper de Maven y los archivos de configuración para que todo funcionara desde la raíz del repositorio. Este commit se ve en el historial como "Migra backend Spring Boot (gms-api) al monorepo".

Configuración del repositorio: agregó el .gitignore para Java, Node y archivos de entorno, evitando subir archivos compilados, dependencias descargadas o contraseñas. Este commit aparece como "Agrega .gitignore para Java, Node y archivos de entorno".

Refactorización del código: implementó la refactorización de MemberService.updateFullProfile() aplicando Extract Method, que fue la mejora más importante de la Fase 5. Este commit aparece como "Implementa refactorización de MemberService.updateFullProfile con Extract Method".

Pruebas del backend: escribió las pruebas de reportes (ReportControllerTest) que completaron la trazabilidad de los 10 requisitos funcionales, y verificó que las 114 pruebas del backend pasaran después de la refactorización.

Métricas: colaboró en la ejecución de las herramientas de medición (Lizard y CK) y en la interpretación de los resultados.

Dayana Motta Camayo

Dayana se enfocó principalmente en el frontend, la infraestructura y la documentación. Su trabajo incluyó:

Migración del frontend, la base de datos y la infraestructura al monorepositorio: fue la encargada de mover el portal de React, las migraciones de Flyway y los archivos de Docker a la nueva estructura. Este commit aparece como "feat: se agrega infra, db, portal".

Ajuste del Docker Compose y los Dockerfiles: corrigió las rutas del docker-compose.yml y del Dockerfile.api para que apuntaran a la nueva estructura del monorepositorio, y verificó que la aplicación levantara correctamente. Este commit aparece como "Ajusta rutas del docker-compose y Dockerfile a la estructura del monorepo".

Documentación de la Fase 1: escribió el documento de requisitos funcionales y no funcionales, y la tabla de trazabilidad inicial. Este commit aparece como "Agrega Fase 1: requisitos funcionales, no funcionales y trazabilidad".

Revisión y aprobación de Pull Requests: en la mayoría de las fases, Dayana fue la encargada de revisar el trabajo de Sebastián y hacer el merge a main, lo cual se ve en el historial como los "Merge pull request" que llevan su nombre.

Scripts de medición y pruebas de carga (Fase 6): colaboró en la creación de los scripts de PowerShell para ejecutar Lizard y CK, y en el script de prueba de carga para el RNF-01. Este commit aparece como "Agrega Fase 6: scripts de medición y pruebas de carga".

Trabajo conjunto

Aunque cada uno tuvo roles distintos, hubo partes del proyecto que hicimos entre los dos:

Arquitectura (Fase 2): definimos juntos el estilo de monolito modular con arquitectura en capas, y Sebastián se encargó de documentarlo con los diagramas.

Métricas (Fases 4 y 5): ejecutamos juntos las herramientas de medición, interpretamos los resultados y decidimos qué refactorización implementar.

Informe final (Fase 6): redactamos el informe entre los dos, revisando cada sección y ajustando el tono para que quedara claro y coherente.

Reflexión sobre el trabajo en equipo

El historial de commits muestra que ambos participamos en todas las fases, aunque con roles distintos. Eso fue importante porque nos permitió revisar el trabajo del otro, detectar errores a tiempo y asegurarnos de que el proyecto cumpliera con todos los requisitos. El uso de ramas por funcionalidad y de Pull Requests con revisión cruzada fue clave para mantener el orden y la calidad del repositorio, y es una práctica que nos llevamos para futuros proyectos.

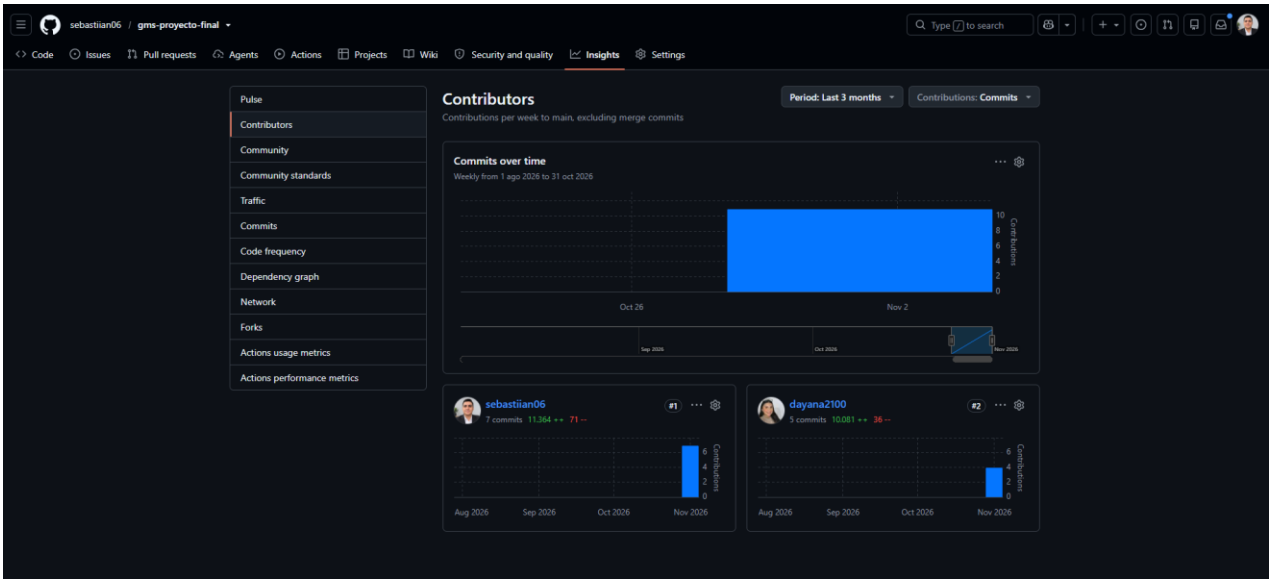


Figura 7. Contribuciones al repositorio.

La imagen muestra los commits y las líneas agregadas por cada integrante, evidenciando una participación equilibrada en el proyecto.

REFERENCIAS Y ENLACE AL REPOSITORIO

Referencias bibliográficas y de herramientas

A continuación presentamos las fuentes, librerías y herramientas que utilizamos para el desarrollo del proyecto y para la aplicación de las métricas. Todas ellas son de uso libre y fueron citadas para cumplir con las reglas de autoría y originalidad del curso.

Frameworks y librerías del sistema:

- Spring Boot. (2024). Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/docs/current/reference/html/

- Spring Security. (2024). Spring Security Reference. https://docs.spring.io/spring-security/reference/
- Meta. (2024). React Documentation. https://react.dev/
- Vite. (2024). Vite Documentation. https://vitejs.dev/
- Tailwind Labs. (2024). Tailwind CSS Documentation. https://tailwindcss.com/docs
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. https://www.postgresql.org/docs/16/
- Redgate. (2024). Flyway Documentation. https://documentation.red-gate.com/flyway
- Docker Inc. (2024). Docker Compose Documentation. https://docs.docker.com/compose/

Herramientas de medición de métricas:

- Yin, T. (2024). Lizard: A code complexity analyzer. GitHub. https://github.com/terryyin/lizard
- Aniche, M. (2024). CK: An open-source tool for calculating Chidamber and Kemerer metrics. GitHub. https://github.com/mauricioaniche/ck

Fundamentos teóricos de las métricas:

- McCabe, T. J. (1976). A Complexity Measure. IEEE Transactions on Software Engineering, SE-2(4), 308-320.
- Chidamber, S. R., & Kemerer, C. F. (1994). A Metrics Suite for Object Oriented Design. IEEE Transactions on Software Engineering, 20(6), 476-493.
- Halstead, M. H. (1977). Elements of Software Science. Elsevier.
- Llanos Mosquera, J. M. (2026). Guía del proyecto final: Métricas de Mantenibilidad y Evolución aplicadas a una Aplicación Real. Curso de Métricas y Estándares de Calidad.

Enlace al repositorio de GitHub

El código fuente completo, la documentación, los resultados de las métricas y los scripts de medición están disponibles en el siguiente repositorio:

Repositorio: https://github.com/sebastiian06/gms-proyecto-final

Estructura del repositorio:

- docs/: documentación del proyecto (requisitos, arquitectura, métricas, informe final).
- src/api/: backend en Spring Boot.
- src/portal/: frontend en React.
- src/db/: migraciones y seeds de la base de datos.
- infra/: archivos de Docker Compose y Dockerfiles.
- metricas/: scripts de medición y resultados en CSV.
- tests/: pruebas adicionales (prueba de carga para el RNF-01).

DECLARACIÓN DE AUTORÍA Y USO DE INTELIGENCIA ARTIFICIAL

En cumplimiento de las reglas académicas del curso, declaramos lo siguiente:

Autoría y originalidad

El código de la aplicación GMS fue desarrollado originalmente por los integrantes del grupo para la asignatura de Desarrollo Fullstack, y posteriormente adaptado para este proyecto de Métricas y Estándares de Calidad con la autorización del docente. Todos los documentos, diagramas, análisis y conclusiones de este informe son producto de nuestro trabajo.

Las librerías, frameworks y herramientas de terceros que utilizamos están citadas en la sección de referencias. No incluimos fragmentos de código de terceros sin citarlos, y las herramientas de medición (Lizard y CK) son de código abierto y se usaron conforme a sus licencias.

Uso de herramientas de inteligencia artificial

Durante el desarrollo de este proyecto utilizamos herramientas de inteligencia artificial como apoyo en las siguientes tareas:

1. Redacción y organización de la documentación: nos apoyamos en un asistente de IA para estructurar los documentos de requisitos, arquitectura, métricas y este informe final, y para revisar la redacción y la claridad de los textos.
2. Ejecución de las herramientas de medición: recibimos orientación sobre cómo instalar y configurar Lizard y CK, y sobre cómo interpretar sus resultados.
3. Análisis de los resultados de las métricas: usamos el asistente para discutir la interpretación de los resultados (por ejemplo, por qué ciertas entidades

JPA tienen un LCOM alto o por qué los servicios tienen un DIT alto) y para validar nuestras conclusiones.

4. Propuesta de refactorización: recibimos orientación sobre la técnica de Extract Method y su efecto esperado sobre las métricas.
5. Redacción del código de los scripts de medición: el asistente nos ayudó a escribir los scripts de PowerShell que ejecutan Lizard y CK, y el script de prueba de carga para el RNF-01.

Lo que no hizo la IA: no generó el código de la aplicación GMS (ese código fue desarrollado por nosotros en la asignatura de Desarrollo Fullstack), no inventó los resultados de las métricas (esos salieron de ejecutar las herramientas sobre nuestro código) y no tomó las decisiones de diseño ni de refactorización (esas fueron nuestras, basadas en el análisis de los resultados).